# Steiner Tree and Traveling Salesman Problem

**Master in Digital Technology**

Student ID: 603927

TU Clausthal

# Motivation

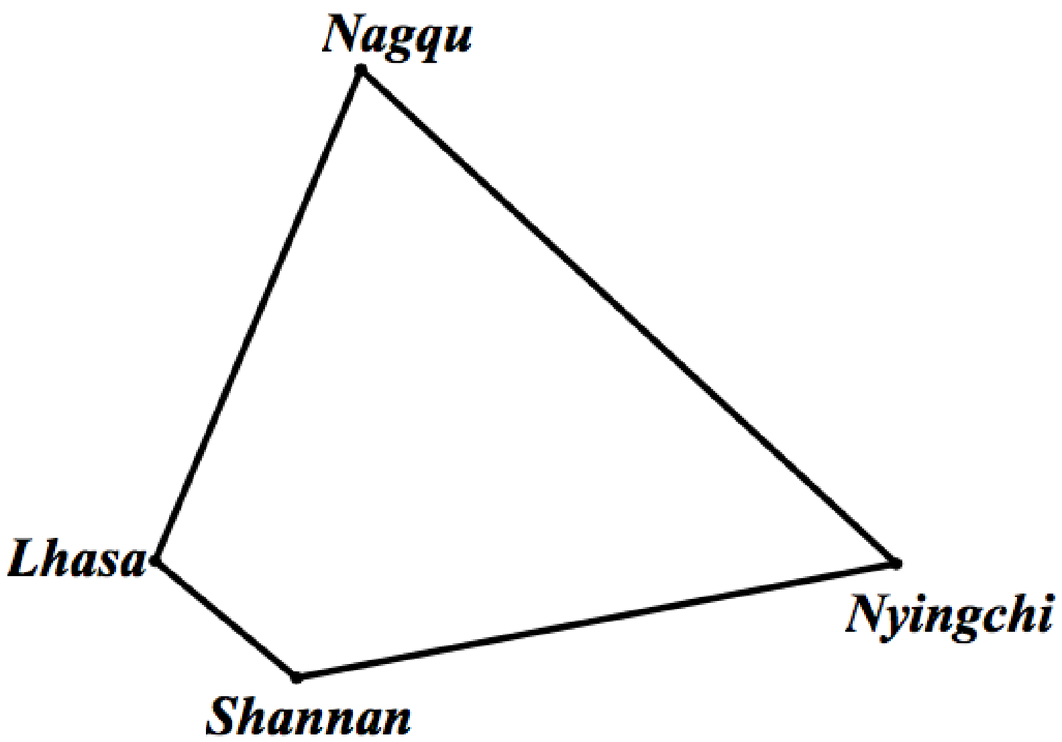
## Why Study These Problems?

- Many real-world problems require connecting locations at minimum cost
  - Examples:
    - Network design (fiber, internet, roads)
    - VLSI chip wiring
    - Biological phylogenetic trees
    - Logistics and route planning
- Two fundamental questions arise:
  - How to connect points cheaply? (Steiner Tree)
  - How to visit all points efficiently? (TSP)
- Exact solutions are computationally infeasible
- We therefore study approximation algorithms



# City Connections Example

## Cities connecting directly




## Cities connected with junction

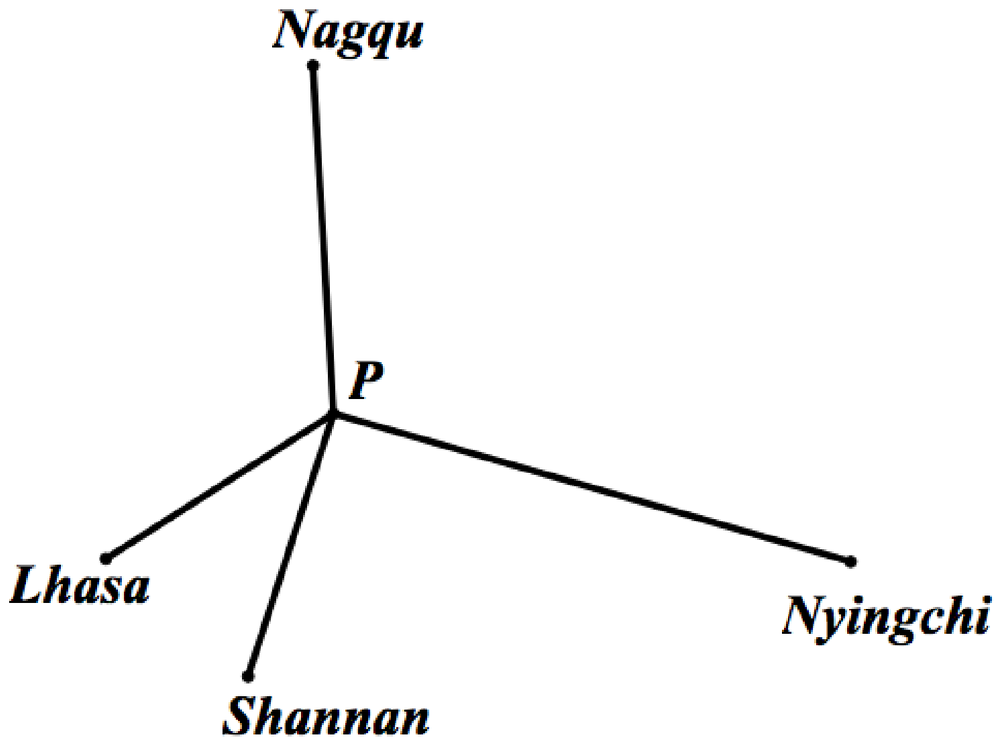



*Figure 2 Two possible arrangements for building Roads*

# The Steiner Tree Problem

Connecting terminals with minimum cost

# Block 1: Problem 3.1(Steiner Tree)

## Concept

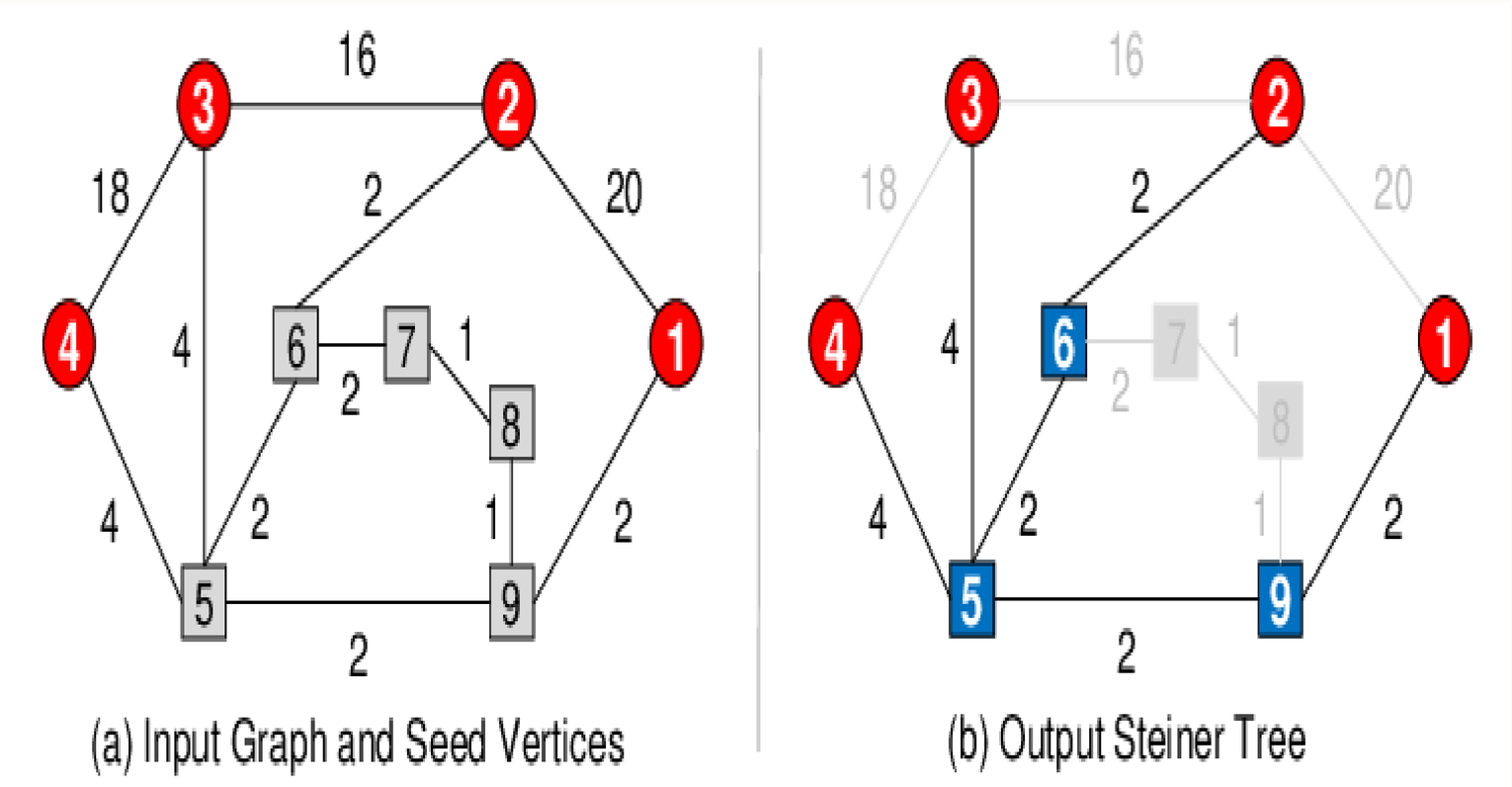
Given:

- An undirected graph
  - $G = (V, E)$
- Each edge has a non-negative cost
- Vertices are split into two groups:
  - Required vertices (terminals)
  - Steiner vertices (optional)

Objective:

- Find a minimum-cost tree that:
  - Contains all required vertices
  - May contain any subset of Steiner vertices

## Diagram: Showing : input graph and Steiner Tree

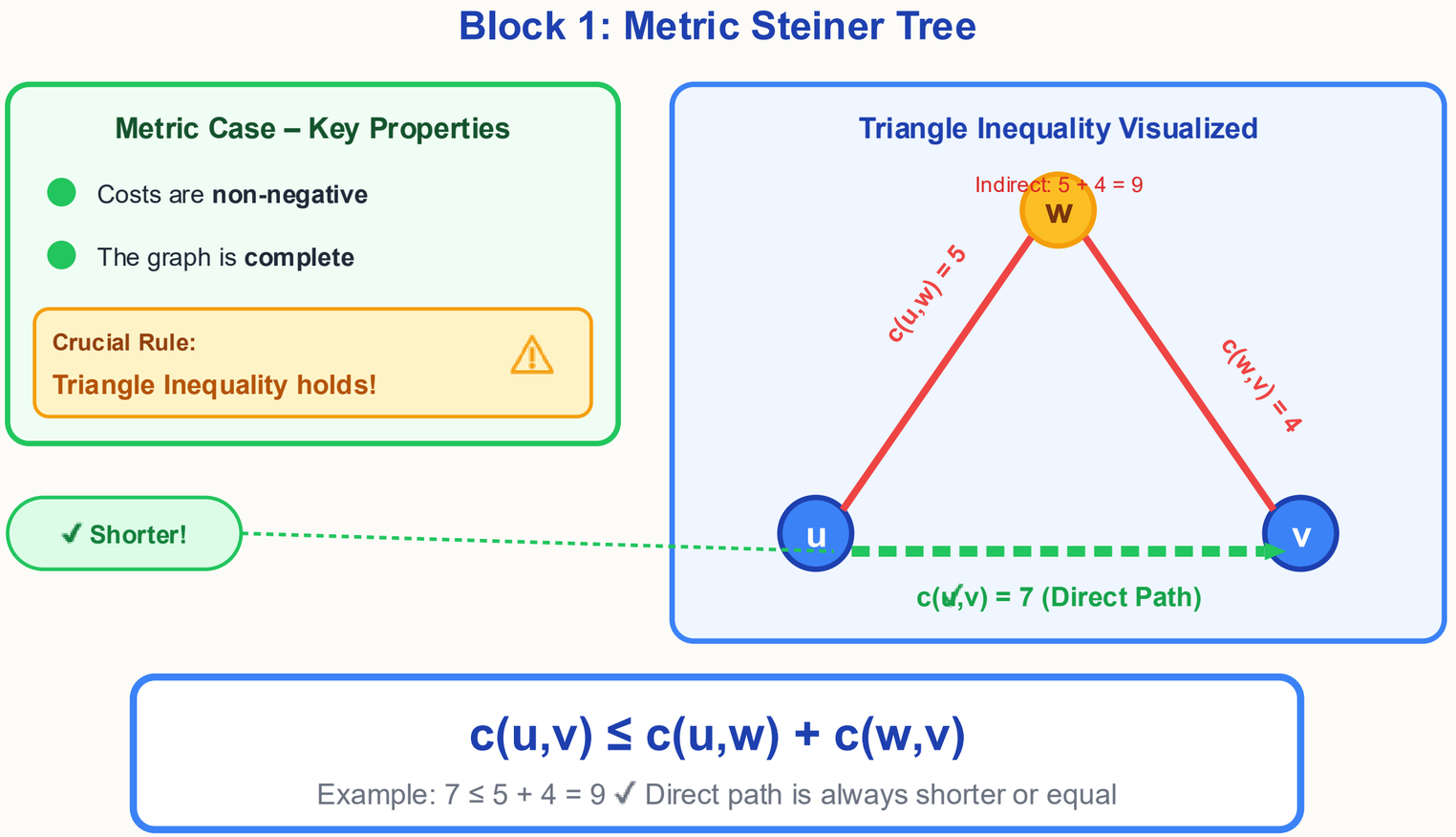


# Block 1: Metric Steiner Tree

## Concept

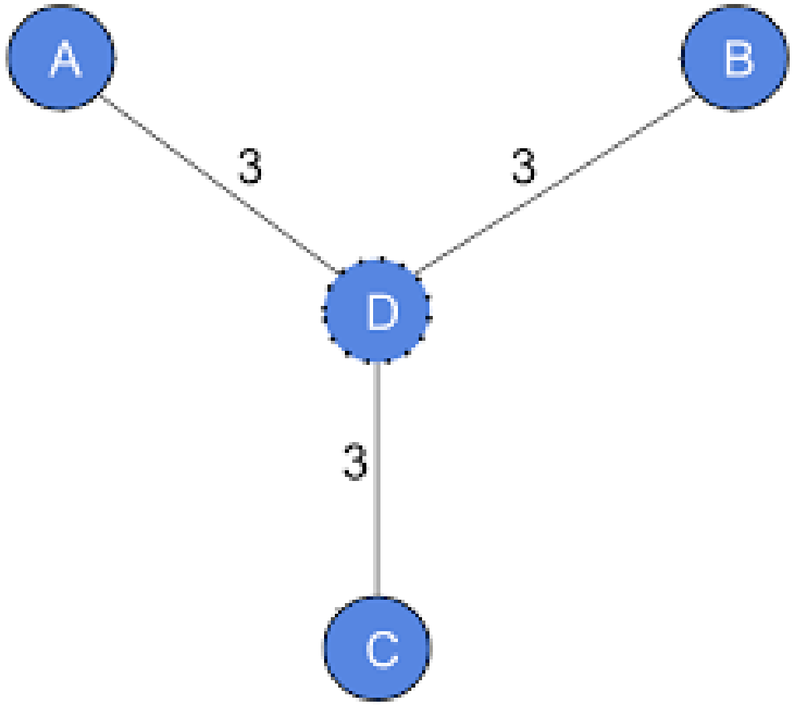
Given:

- We focus on the Metric Case.
- Costs are non-negative.
- The goal is to find a minimum-cost tree spanning all terminal nodes.
- Crucial Rule: Triangle Inequality holds.

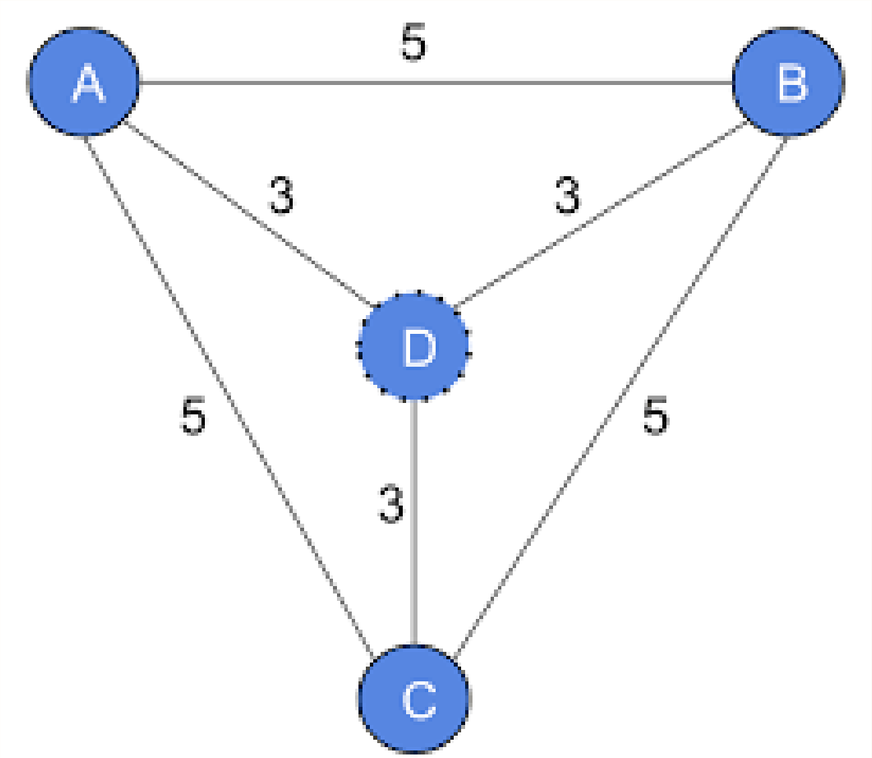


# Steiner Tree and Complete Graph

## Steiner Tree



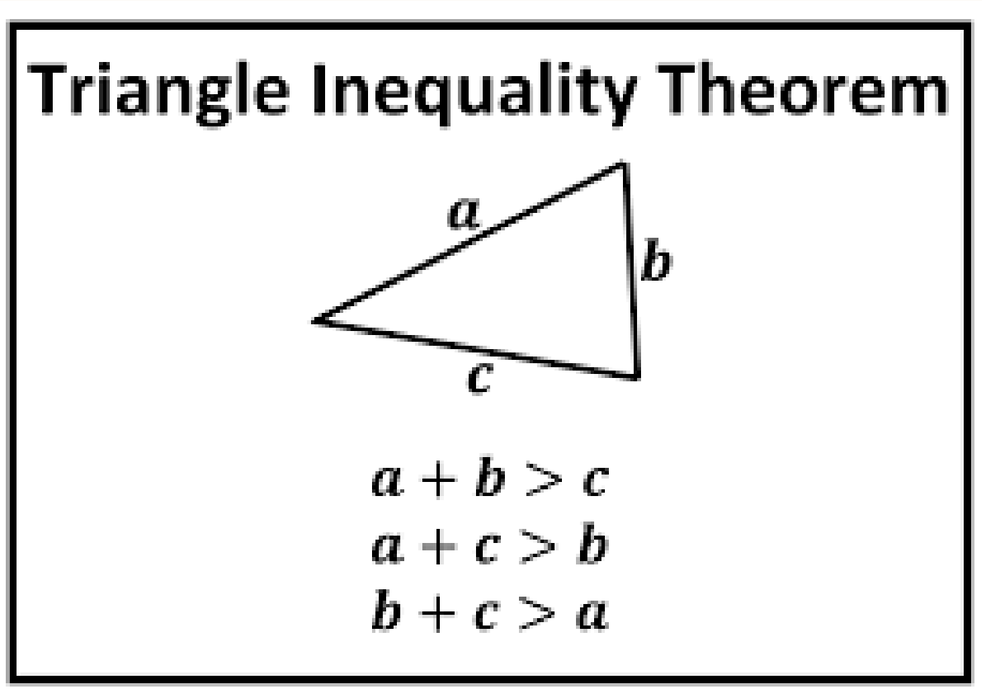

## Complete Graph



# Triangle Inequality

- The core of the problem lies in how edge costs behave
- We focus on instances where:
  - The graph is complete
  - Edge costs are non-negative
  - Triangle inequality holds:
    - cost(u,v) ≤ cost(u,w) + cost(w,v)
- Triangle inequality ensures distances behave realistically
- In a metric world, the shortest distance between two points is a direct line.

## Trainagle Inequality Theorem



# Block 1: Theorem 3.2: The Reduction

- **Theorem:**

**There is an approximation-factor preserving reduction from Steiner Tree to Metric Steiner Tree.**

Reduction: Steiner Tree → Metric Steiner Tree

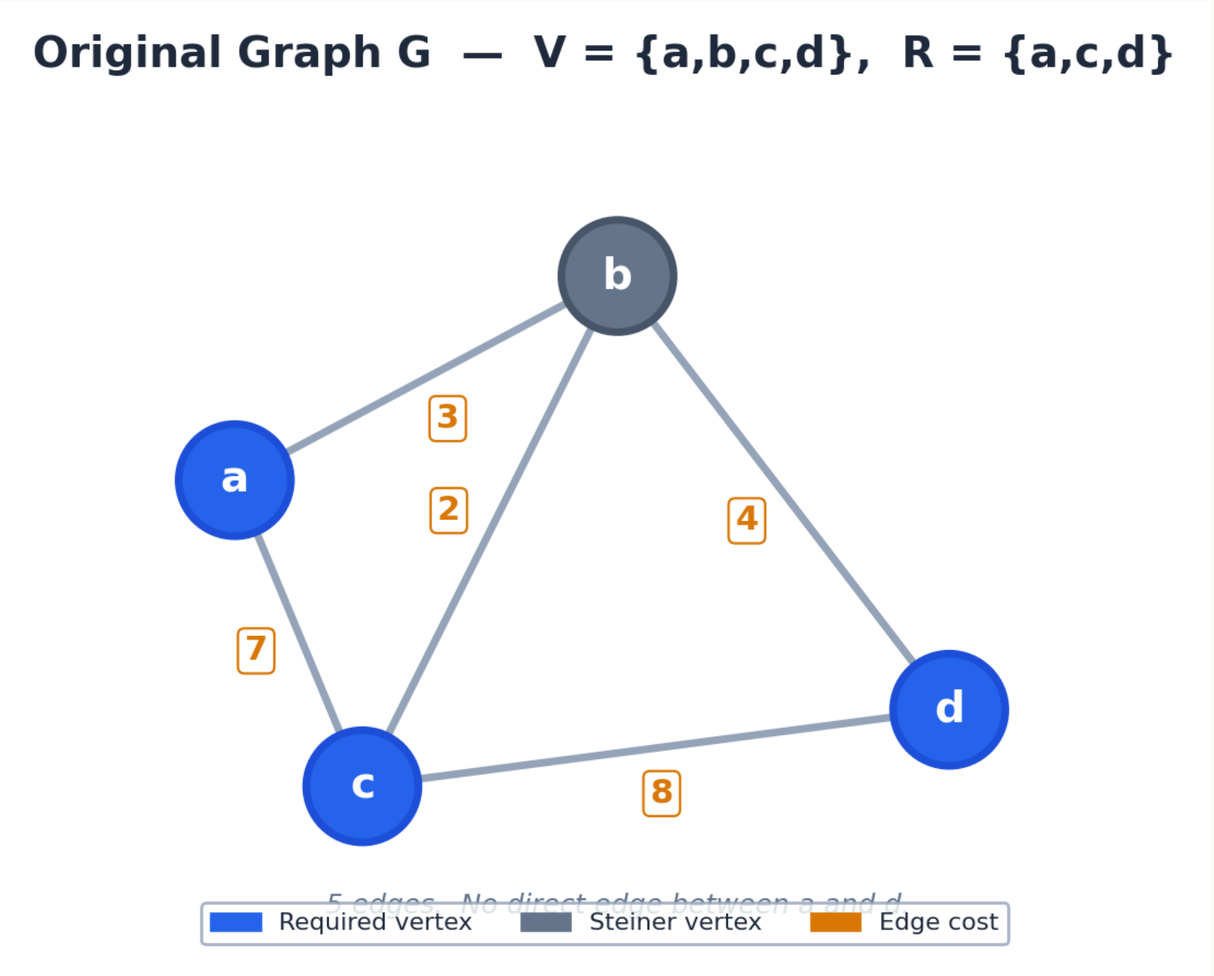
# Block 1: Theorem 3.2: Proof

**Instance I: Graph G = (V, E)**

V = {a, b, c, d}  Required R = {a, c, d}  Steiner = {b}

Example:  V = {a, b, c, d},  R = {a, c, d} (Required), Steiner = {b}

Edges:  a-b(3),  b-c(2),  b-d(4),  a-c(7),  c-d(8)



# Block 1: Theorem 3.2: Proof

## Construction: Shortest Path for Every Pair

Pair (a, b)
- Direct edge a – b:  cost = 3
- Via c: a → c → b = 7+2 = 9

-> cost'(a,b) = min(3, 9) = 3

Pair (a, d)
- No direct edge a–d in G
- Via b: a→b→d = 3+4 = 7
- Via c,b: a→c→b→d = 7+2+4 = 13

⟶ cost'(a,d) = 7  (new edge!)

Pair (b, d)
- Direct edge b–d: cost = 4
- Via c: b→c→d = 2+8 = 10

⟶ cost'(b,d) = min(4, 10) = 4

Pair (a, c)
- Direct edge a–c: cost = 7
- Via b: a→b→c = 3+2 = 5

⟶ cost'(a,c) = min(7, 5) = 5  ★ shorter!

Pair (b, c)
- Direct edge b–c: cost = 2
- Via a: b→a→c = 3+7 = 10

⟶ cost'(b,c) = min(2, 10) = 2

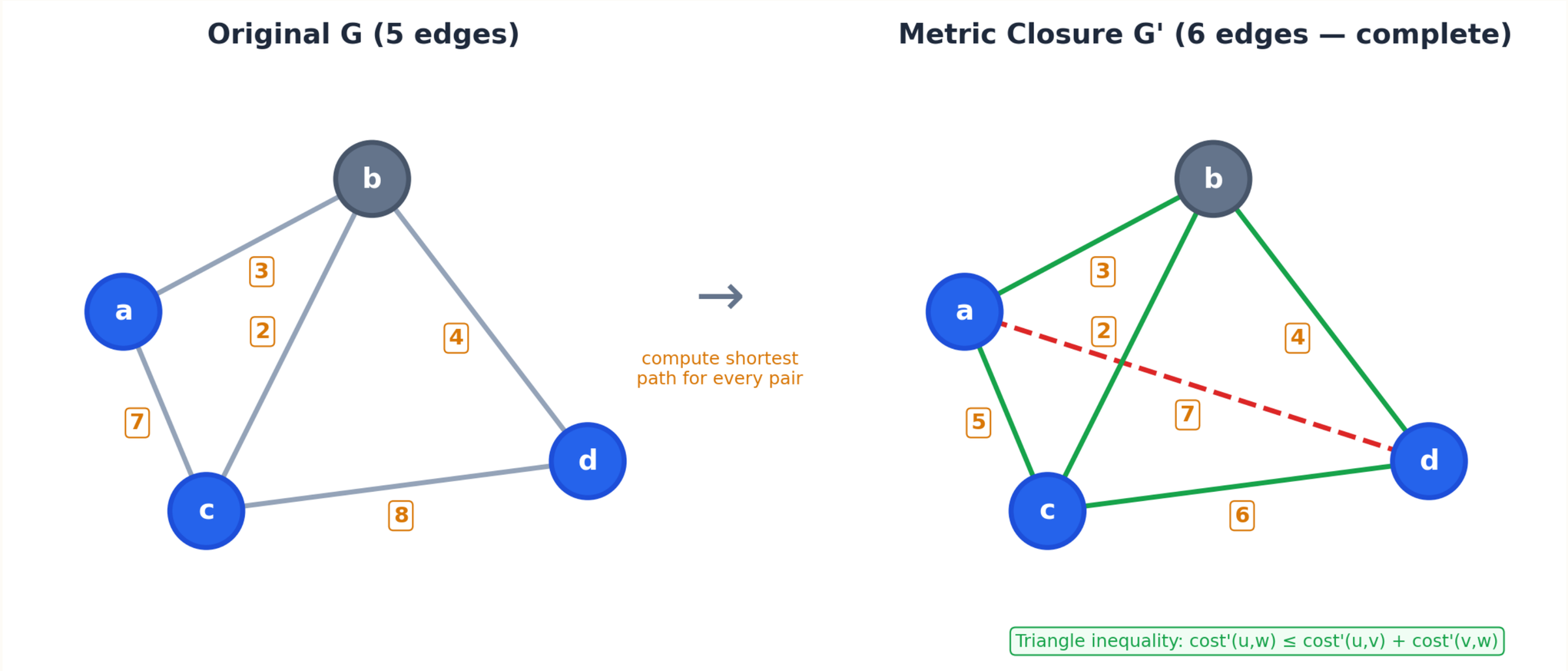
Pair (c, d)
- Direct edge c–d: cost = 8
- Via b: c→b→d = 2+4 = 6

⟶ cost'(c,d) = min(8, 6) = 6  ★ shorter!

# Block 1: Theorem 3.2: Proof

## Metric Closure G' (Complete Graph)

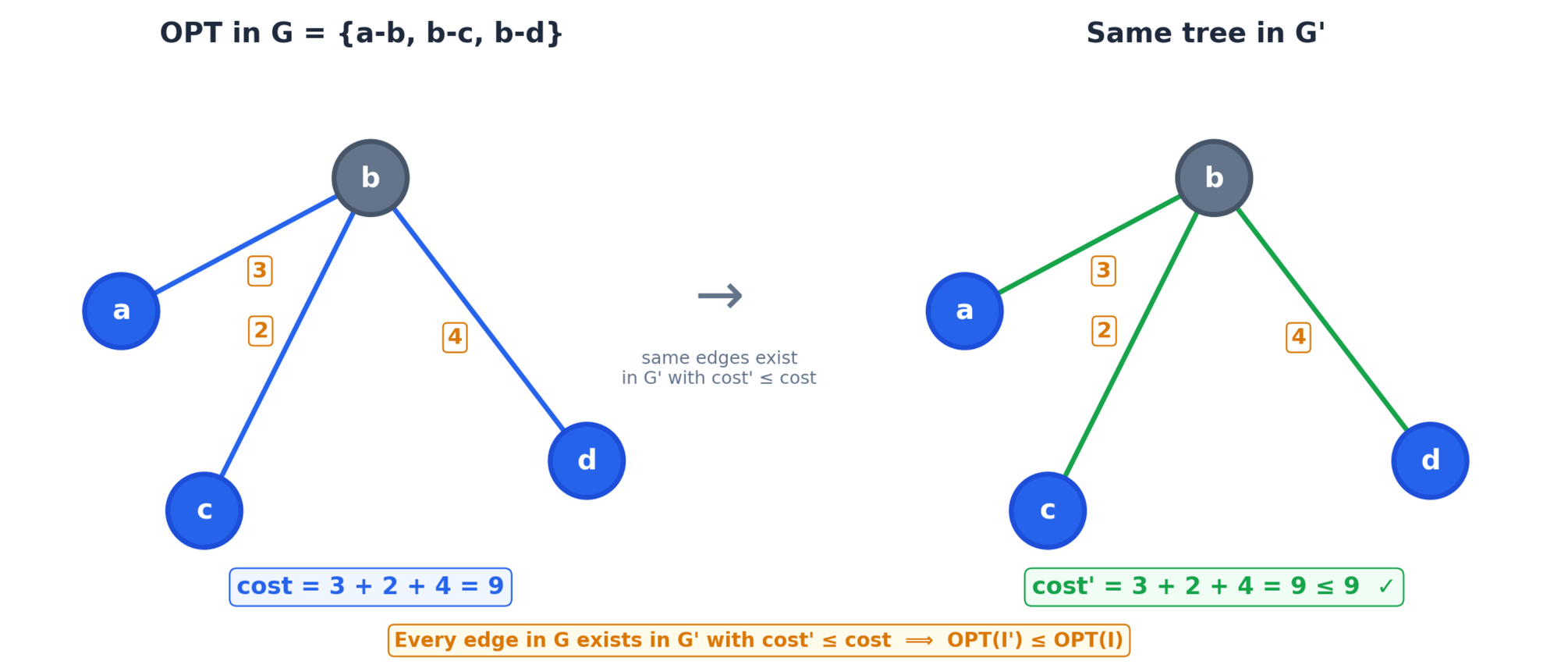
G' has C(4,2)=6 edges.  cost'(u,v)=dist_G(u,v).  R'=R={a,c,d}.



# Block 1: Theorem 3.2: Proof

## Direction 1: OPT(I') ≤ OPT(I)

For any edge (u,v) ∈ E, cost in G' ≤ cost in G ⟹ OPT(I') ≤ OPT(I).

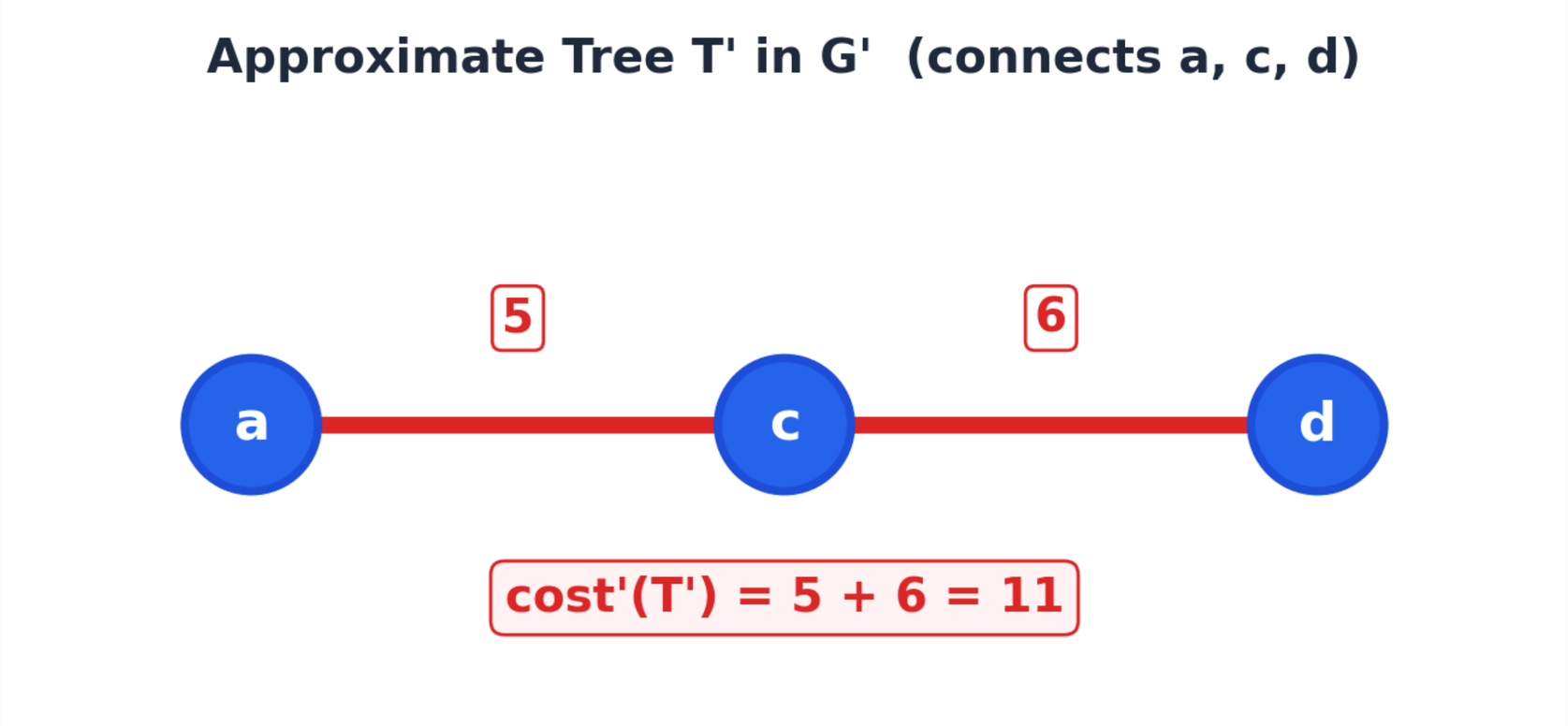


# Block 1: Theorem 3.2: Proof

## Direction 2, Step A: Approx Tree T' in G'

Suppose an α-approximation gives T' = {a-c, c-d} in G'.

**cost'(T') = 5 + 6 = 11  (connects all required vertices a, c, d)**



# Block 1: Theorem 3.2: Proof

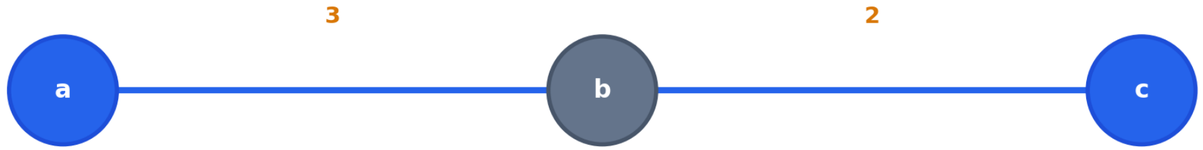
## Direction 2, Step B: Replace Edges by Shortest Paths

Replace each edge of T' by its corresponding shortest path in G:

Edge (a,c) in T' — cost' = 5



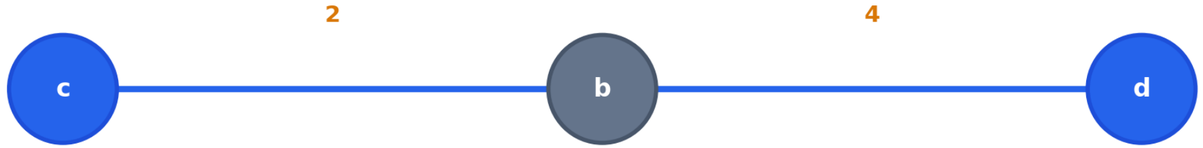

Shortest path a→b→c in G — cost = 3+2 = 5




Edge (c,d) in T' — cost' = 6



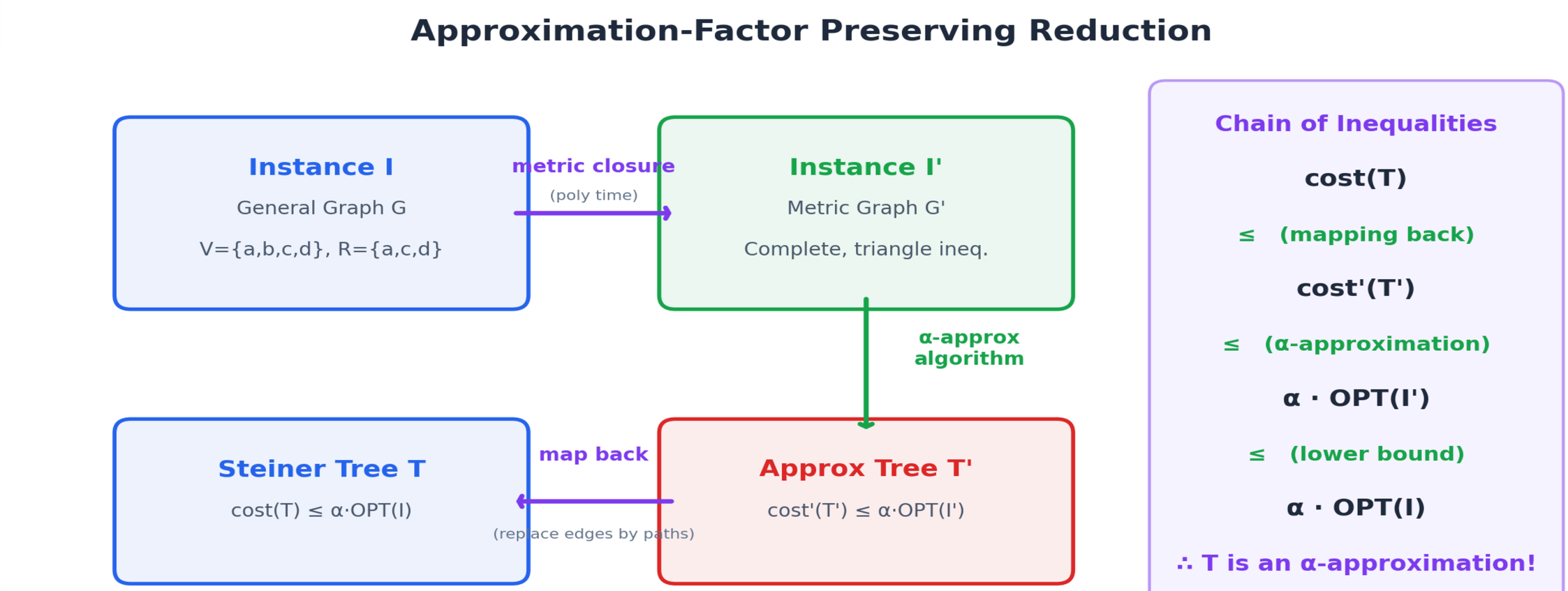

Shortest path c→b→d in G — cost = 2+4 = 6




Union: {a-b(3), b-c(2)} ∪ {c-b(2), b-d(4)} = {a-b, b-c, b-d}  (duplicate b-c removed)

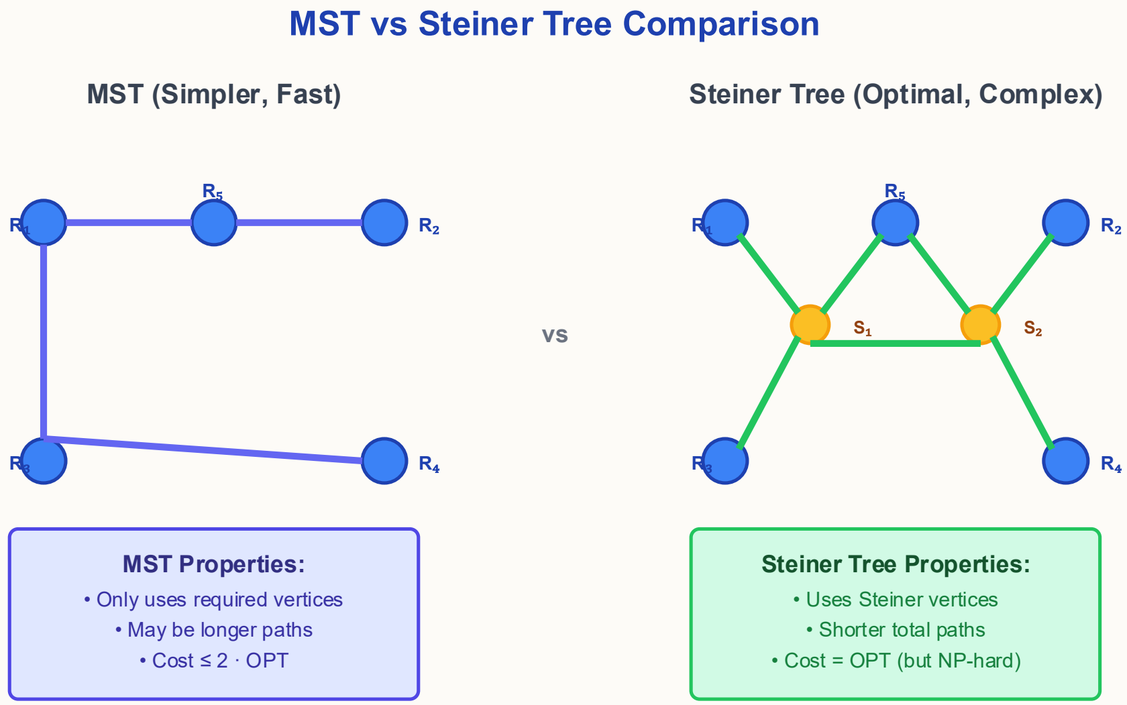
# Block 1: Theorem 3.2: Proof : Conclusion

## Factor-Preserving Reduction



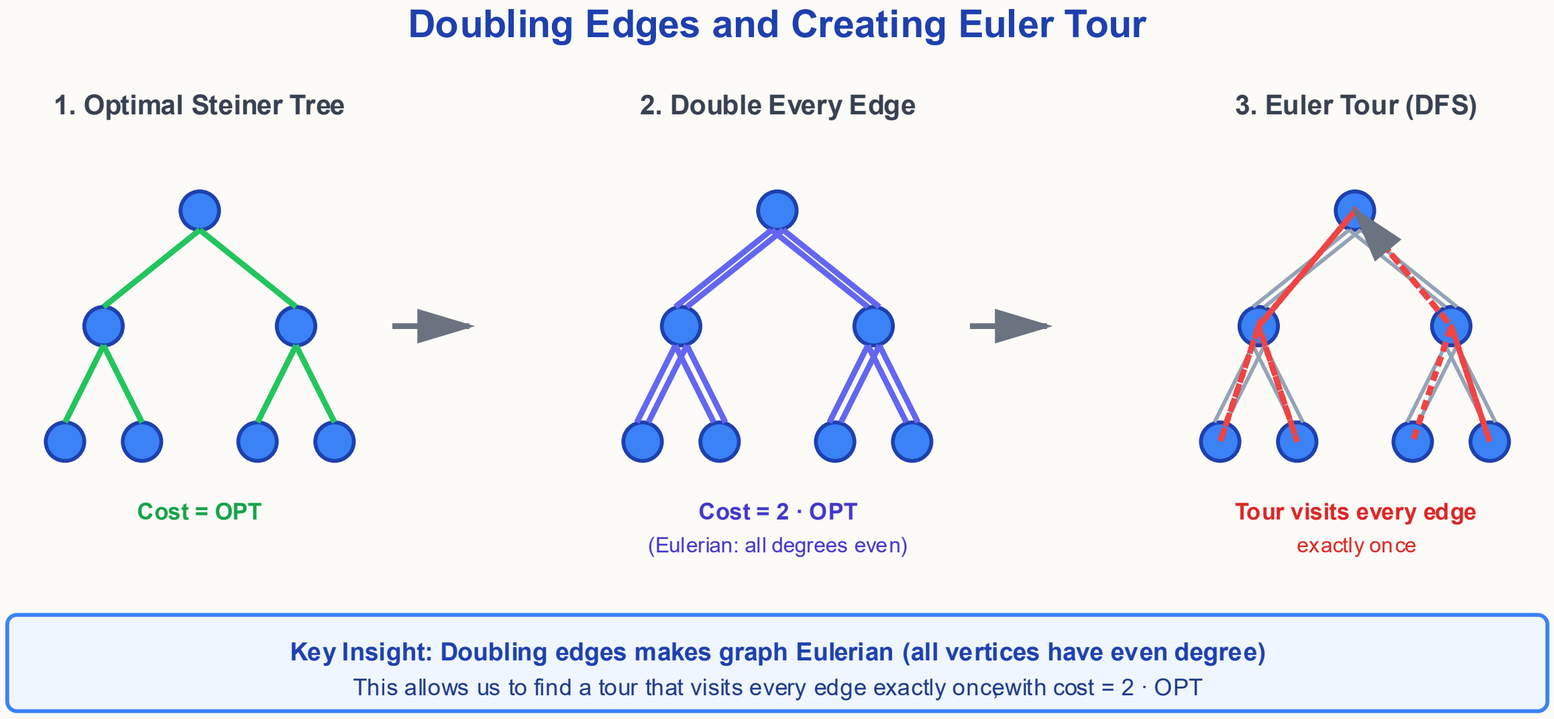
# Block 2: MST-based Algorithm

- Concept:
  - Let R be the set of Required vertices.
  - An MST on R is a feasible solution (it connects everything we need).
- The Comparison:
  - Finding an MST is in P (Fast).
  - Finding a Metric Steiner Tree is NP-hard (Slow).
  - We use the MST as a fast substitute, even if it's slightly more expensive.
- Theorem 3.3: The cost of an MST on R is within 2 · OPT.



# Block 2: Proof of Theorem 3.3 (Part 1) Doubling and Touring

- Step 1: Double the Edges
  - Take an optimal Steiner tree (Cost = $OPT$).
  - Double every edge (Cost = 2 . $OPT$).
  - This creates a Eulerian graph (every node has even degree).
- Step 2: Find a Euler Tour
  - Use DFS to find a tour that visits every vertex and uses every edge exactly once.
  - Tour Cost = 2 . OPT.



# Block 2: Proof of Theorem 3.3 (Part 2)
# Short-cutting

- Step 3: Short-cut to Hamiltonian Cycle
  - Traverse the Euler tour.
  - If a Steiner vertex or a previously visited vertex appears,

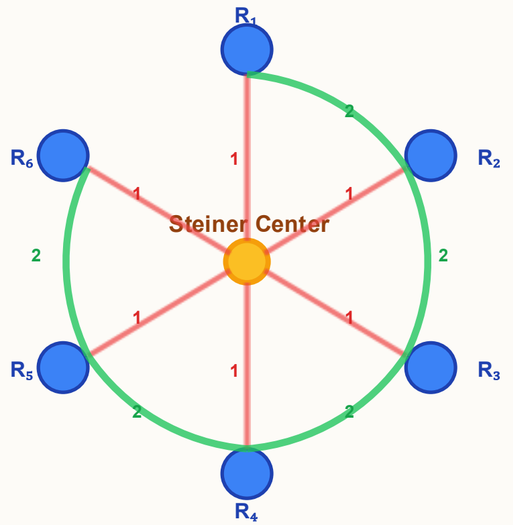
    skip it and go to the next unvisited required vertex.
- The Logic (Triangle Inequality):
  - Because the graph is metric, skipping points (short-cutting)

    never increases the cost.
  - We now have a Hamiltonian Cycle on R with cost $\leq 2 \cdot OPT$.
- Final Step:
  - Delete one edge from this cycle.
  - We get a path on R with cost $\leq 2 \cdot OPT$.

# Block 2: Example

- The Scenario:
  - *n* required vertices on a circle.
  - 1 Steiner vertex in the dead center.
  - Distance from center to any required vertex = 1.
  - Distance between any two required vertices = 2.
- The Math:
  - Optimal (OPT): Connect all n nodes to the center Steiner node. Cost = n.
  - MST on R: Since we can't use the center, we connect nodes along the perimeter. Cost = 2(n-1).
- The Ratio:
  - As *n* gets larger, the ratio 2(n-1)/ n approaches 2.

**Star Graph: Tight Example for Factor 2**




**Optimal Steiner Tree (OPT)**
n=6 terminals, cost = n = 6

**MST on Required Vertices**
Perimeter path, cost = 2(n-1) = 10

**Ratio = MST / OPT = 2(n-1) / n**

For n=6: 10/6 ≈ 1.67 As n → ∞: ratio → 2

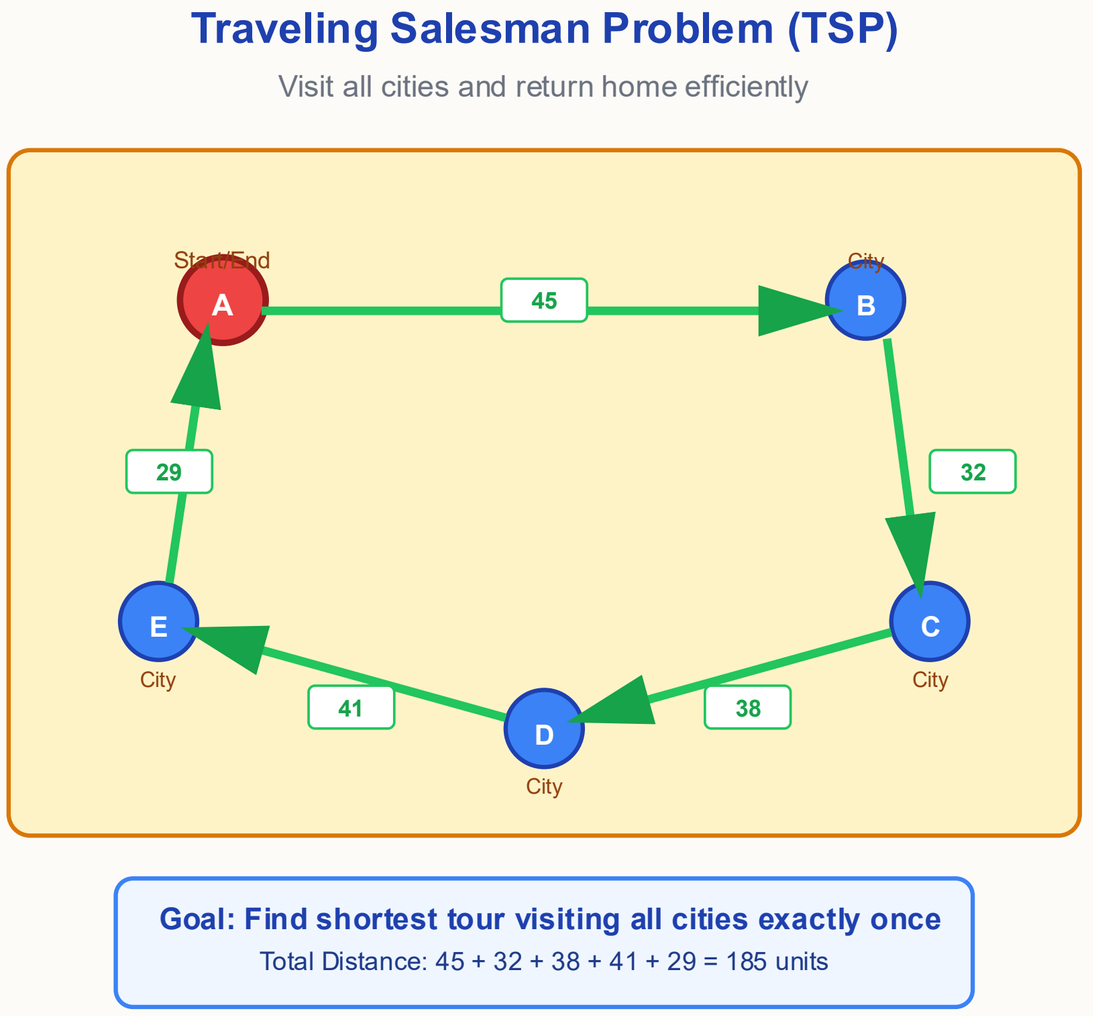
*This proves factor 2 is tight (Example 3.4, Vazirani)*

# The Traveling Salesman Problem

optimization challenge that asks for the shortest possible route to visit a set of cities exactly once and return to the origin

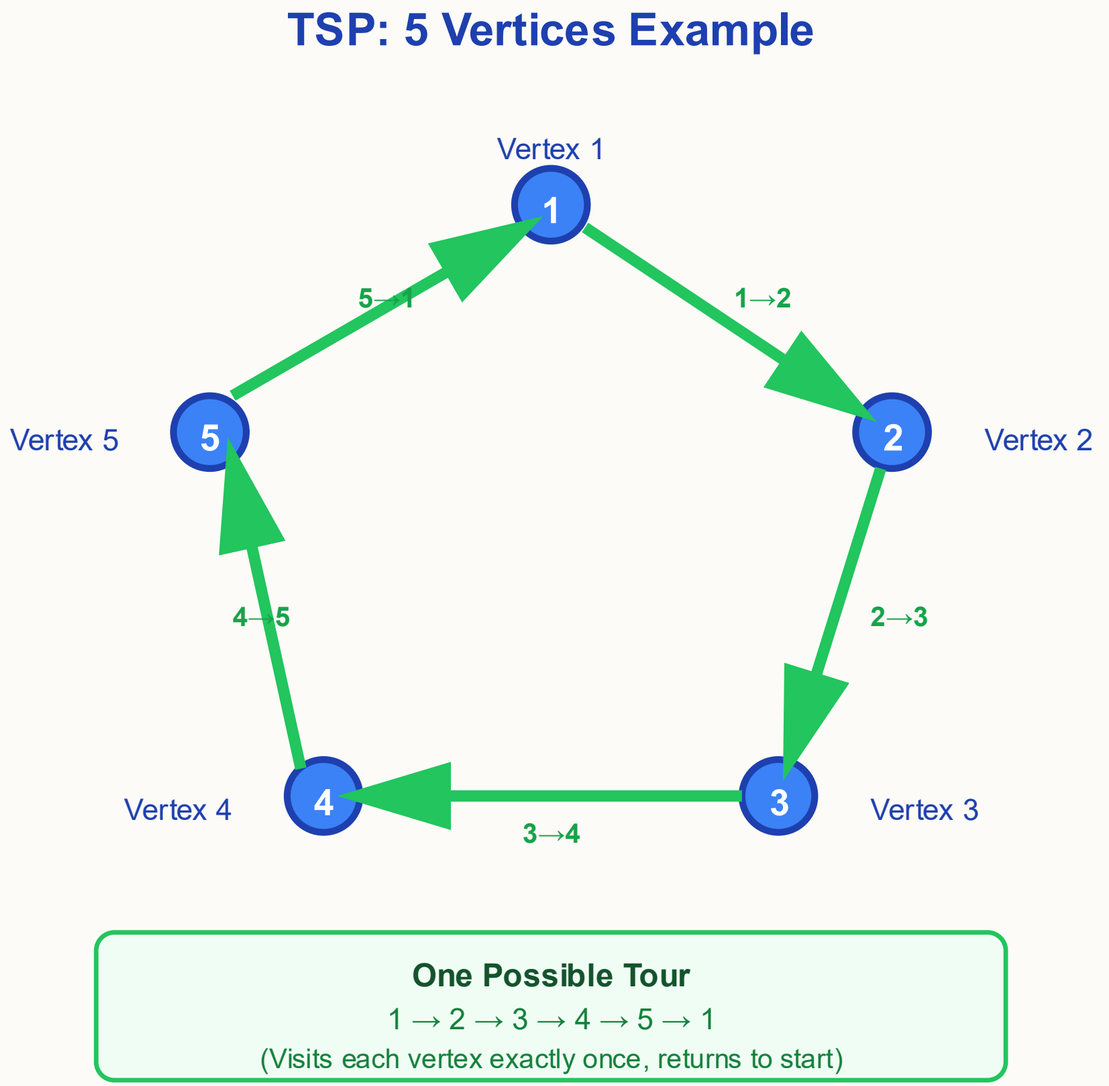
# What is TSP?

- What is TSP?
  - Imagine a delivery driver who needs to visit every city atleast once and return home.
- The Goal:
  - Find the absolute shortest path that visits every city exactly once and returns to the start.
- Why it's hard:
  - The number of possible paths grows incredibly fast (Factorial!).



# Problem 3.5 - The Traveling Salesman Problem

- Definition:
  - Given a complete graph with non-negative edge costs.
  - Find a minimum cost cycle visiting every vertex exactly once.
- The Hard Truth:
  - In its full generality, TSP cannot be approximated at all (assuming P ≠ NP).



# Theorem 3.6 - Hardness of General TSP

- Statement:
  - For any polynomial time computable function $\alpha(n)$,

    TSP cannot be approximated within a factor of $\alpha(n)$,

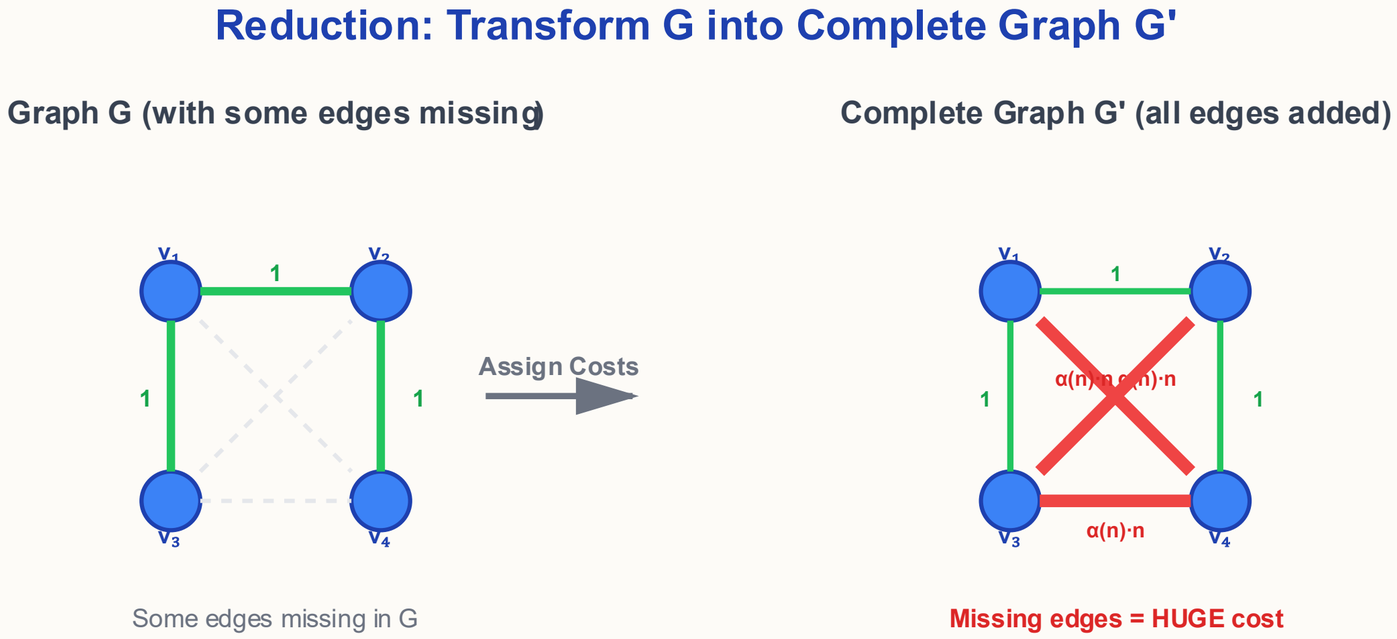
    unless P = NP.
- Simple Explanation:
  - Even if you ask for an answer within a factor of 100

    or 1,000,000, it's still impossible to guarantee.

# Proof of Theorem 3.6 (Part 1) - The Reduction

- The Goal: Show that approximating TSP is as hard as the Hamiltonian Cycle problem.
- The Setup:
  - Transform graph G into a complete graph G'.
- Assigning Weights:
  - If edge exists in G is to Cost is 1.
  - If edge NOT in G is to Cost is α(n) . n (Huge!).

## Reduction: Transform G into Complete Graph G'



# Proof of Theorem 3.6 (Part 2) - The Gap

- Case 1: Hamiltonian Cycle exists

  Tour uses only "1" edges. Total Cost = n.

- Case 2: No Hamiltonian Cycle exists

  Tour must use at least one "missing" edge.

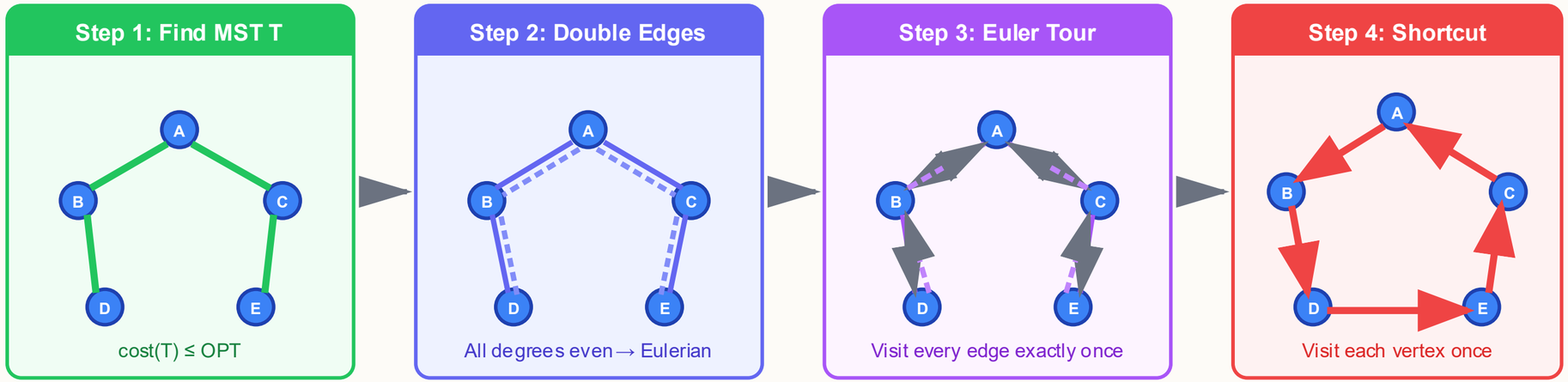
  $$\text{Total Cost} > \alpha(n) \cdot n.$$

- Conclusion: An approximation algorithm would easily see the "Gap" and solve an NP-hard problem.

# Algorithm 3.7 - Metric TSP (Factor 2)

- Find an MST $T$ of the graph.
- Double every edge of the MST (makes it Eulerian).
- Find a Euler tour $T$.
- Short-cut the tour to visit each city once.

## Algorithm 3.7: Metric TSP — Factor 2 Approximation




### Algorithm 3.7 — Metric TSP (Factor 2) [Vazirani, Ch. 3.2.1]

1. Find a **Minimum Spanning Tree (MST)**, T, of graph G.
   *Lower bound: cost(T) ≤ OPT, since deleting any edge of optimal tour gives a spanning tree.*
2. **Double every edge** of the MST to obtain an Eulerian graph.
   *Every vertex now has even degree → Euler tour guaranteed to exist.*
3. Find an **Euler tour** of this graph (e.g., via DFS traversal).
   *Traverse every edge exactly once. Cost of tour = 2 · cost(T).*
4. **Shortcut:** Output the tour visiting vertices in order of first appearance.

✓ cost(C) ≤ 2 · OPT

# Theorem 3.8 - Factor 2 Guarantee

- The Statement: Algorithm 3.7 is a factor 2 approximation algorithm for metric TSP.
- The Meaning:
  - Our final tour cost will never be more than twice the optimal tour cost.

**Factor 2 Guarantee**

$$\text{Cost(Tour)} \leq 2 \cdot \text{OPT}$$

Our algorithm's tour cost is at most twice the optimal tour cost

**2-Approximation Algorithm**

Fast, reliable, and guaranteed to be within factor 2 of optimal

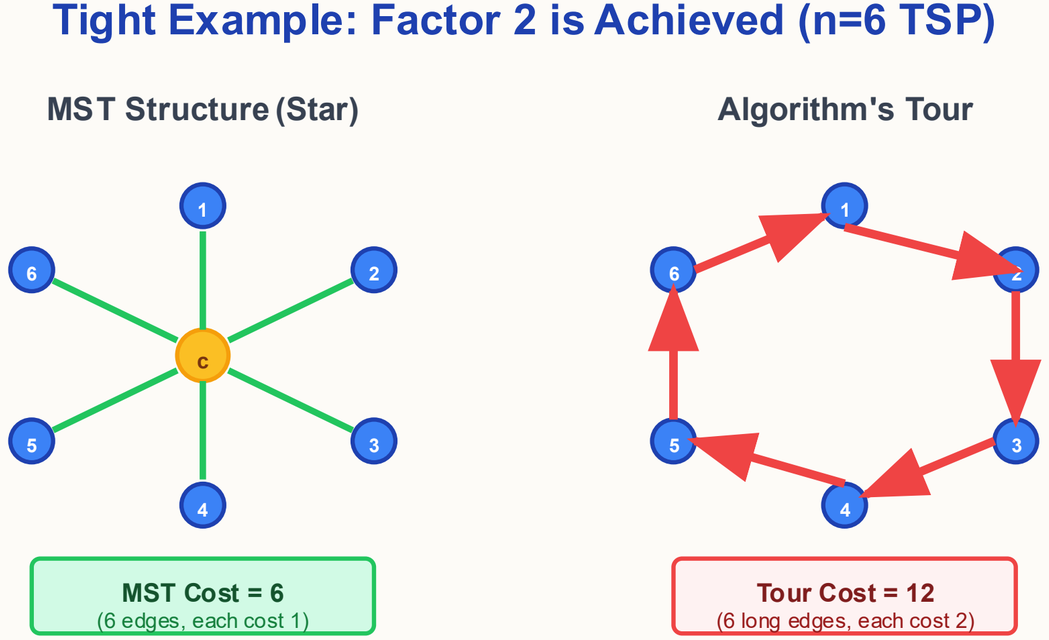
# Proof of Theorem 3.8

- Proof: **Cost** $\leq 2$ OPT:
  - Lower Bound: cost(MST) $\leq$ OPT (Removing one edge from the optimal tour gives a tree).
  - Doubling: The Euler tour T uses each edge twice: cost(T) = 2 . cost(MST).
  - Triangle Inequality: Short-cutting only reduces the cost: cost(Final) $\leq$ cost(T).
- Result:

  cost(Final) $\leq 2$ . cost(MST) $\leq 2$ . OPT.

# Example - Tight Case for Factor 2

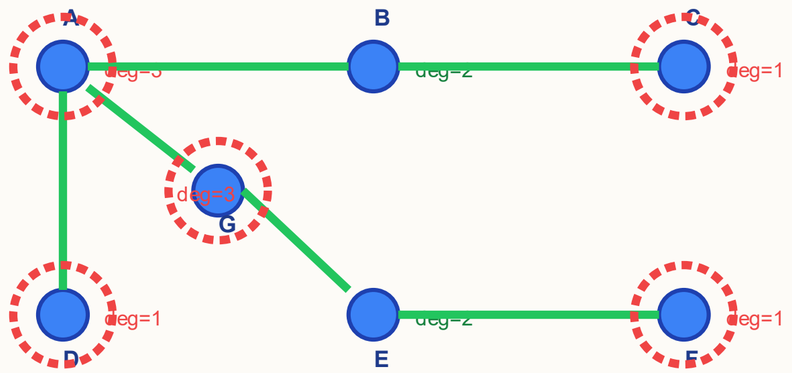
- Scenario:
  - *n* vertices. Edges cost 1 or 2.
  - Optimal tour uses only "1" edges. Cost = n.
  - The algorithm picks a poor Euler tour order.
  - Final Cost = 2n - 2.
- Conclusion: This shows that factor 2 is the best guarantee we can give for this specific MST method.



# Improving the Factor - Why Double Everything?

- The Problem: Doubling every edge is "wasteful."
- The Insight:
  - We only need to make every vertex have an even degree to find an Euler tour.
  - In an MST, only some vertices have odd degrees.
  - We only need to fix those specific vertices.

## MST with Odd-Degree Vertices Highlighted




**Degree Count**

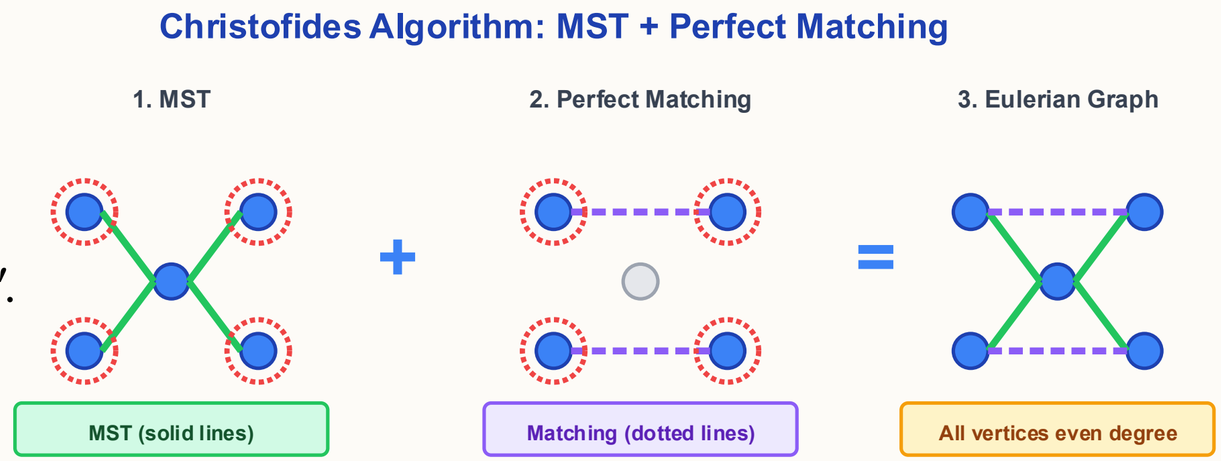
**Even Degree:** B(2), E(2) — Total: 2 vertices

**Odd Degree:** A(3), C(1), D(1), F(1), G(3) — Total: 5 vertices

*Note: Any MST has an even number of odd-degree vertices*

# Algorithm 3.10 - Metric TSP (Factor 3/2)

- Steps (Christofides' Algorithm):
  - Find an MST $T$.
  - Find all odd-degree vertices V'.
  - Find a Minimum Cost Perfect Matching $M$ on $V'$.
  - Combine $M + T$ to get an Eulerian graph.
  - Find Euler tour and short-cut.




**Christofides Algorithm (Factor 3/2)**

**Step 1:** Find MST T of the graph (shown in green solid lines)

**Step 2:** Find odd-degree vertices in T (marked with red dashed circles)

**Step 3:** Find minimum-cost perfect matching M on odd-degree vertices (purple dotted lines)

**Step 4:** Combine T ∪ M → Eulerian graph → Find Euler tour → Shortcut to Hamiltonian cycle

# Lemma 3.11 - Bound on Matching

- The Statement: Let V' be the set of odd-degree vertices.

  Let M be the minimum cost perfect matching on V'. Then cost(M) ≤ OPT/2.
- Key Idea: The matching is "cheap" because it only connects half the nodes from an optimal cycle.

## Matching Cost Bound

$$\text{cost}(M) \leq 0.5 \cdot \text{OPT}$$

Matching cost is at most half the optimal tour cost

**Key Insight**

The optimal TSP tour visits odd-degree vertices and forms a cycle.

This cycle can be split into 2 matchings, so one matching costs ≤ OPT/2.

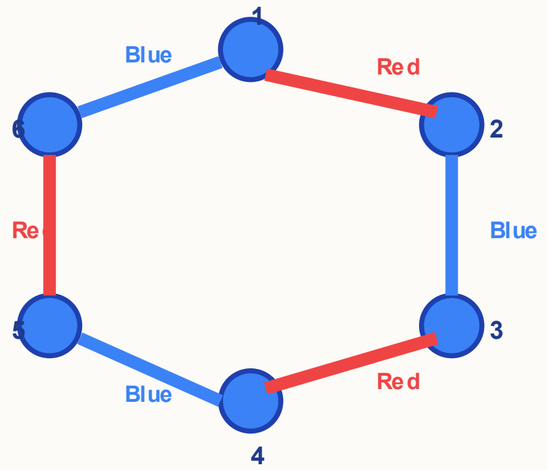
# Proof of Lemma 3.11

- Logic:
  - Take the optimal tour OPT.
  - Short-cut it to visit only the odd vertices (V'). This is a cycle.
  - This cycle is a union of two perfect matchings (alternating edges).
  - By Triangle Inequality, cost(Cycle) $\leq$ OPT.
  - The cheaper of the two matchings must be $\leq$ cost(Cycle)/2 $\leq$ OPT/2.

# Proof of Lemma 3.11

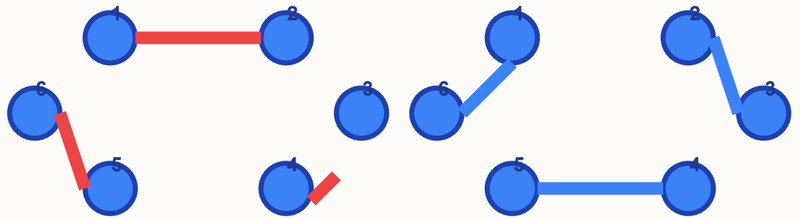
## Cycle Splitting: One Cycle → Two Perfect Matchings

**Original Cycle (Hamiltonian)**




Split

**Matching 1 (Red Edges)** **Matching 2 (Blue Edges)**



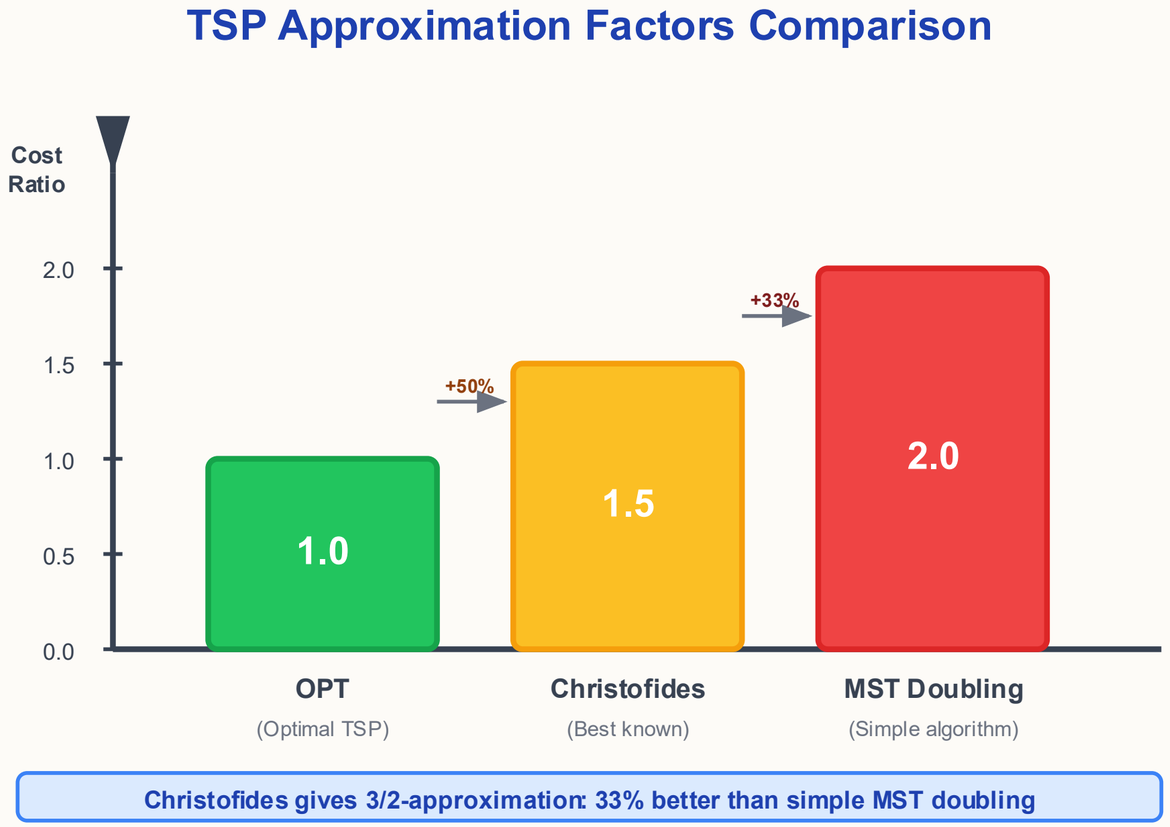
*Edges alternate: Red-Blue-Red-Blue-Red-Blue*

**Key Property: Any cycle with even number of vertices can be split into 2 perfect matchings**

Cost(Cycle) = Cost(Matching 1) + Cost(Matching 2), therefore min matching ≤ Cost(Cycle)/2

# Theorem 3.12 - The 3/2 Factor

- The Analysis:
  - Final Cost = cost(MST) + cost(Matching)
  - cost(MST) ≤ OPT
  - cost(Matching) ≤ 0.5 . OPT
  - Total ≤ 1.5 . OPT
- Conclusion: Christofides' algorithm is a 3/2-approximation.



# Summary & Conclusion

- Key Takeaways:
  - Metric version is the "core" of both Steiner and TSP.
  - Steiner Tree: MST gives factor 2.
  - TSP: MST gives factor 2; Matching gives factor 1.5.

# Questions?

Feel free to ask about the algorithms or proofs.

# Thank You

For Your

Attention

Master in Digital Technology
TU Clausthal

Haris Masood

Special Thanks to: Mr. Bin Sun